NOUN Smart Lecture Reminder

A Progressive Web App (PWA) that gives NOUN (National Open University of Nigeria) students personalised push notifications for their Zoom lectures, TMA deadlines, and facilitator announcements replacing the manual process of checking the eLearn portal every day.

Stack: React + Vite · Supabase (Auth + Postgres + Realtime + Edge Functions + Cron) · Netlify (Static Hosting) · Web Push API (VAPID) · Tailwind CSS · PDF.js · date-fns · Framer Motion (via `motion/react`) · vite-plugin-pwa

---

Table of Contents

1. Project Setup
2. Environment Variables
3. Supabase Database Setup
4. Seeding the Admin Account
5. VAPID Keys Push Notifications
6. Running Locally
7. Netlify Deployment
8. How Each Role Operates the App
    - Admin
    - Lecturer
    - Student
9. Push Notification Flow
10. PWA Installation
11. Folder Structure
12. Known Limitations

---

1. Project Setup

```
# Clone the repository
git clone <your-repo-url>
cd smart-lecture-reminder

# Install frontend dependencies (includes the Supabase CLI as a devDependency)
npm install
```

```
# Link this repo to your Supabase project (one-time)
npx supabase login
npx supabase link --project-ref <your-project-ref>
```

---

2. Environment Variables

Create a `.env` file in the project root (never commit this file):

```
# Supabase
VITE_SUPABASE_URL=https://xxxxxxxxxxx.supabase.co
VITE_SUPABASE_ANON_KEY=<your-supabase-anon-key>

# Web Push (VAPID)  see Section 5 for how to generate
VITE_VAPID_PUBLIC_KEY=<your-vapid-public-key>
```

For the Supabase Edge Functions (set once via the CLI — not stored in any `.env` file, and not read by the Vite app):

```
npx supabase secrets set \
  VAPID_PUBLIC_KEY=<your-vapid-public-key> \
  VAPID_PRIVATE_KEY=<your-vapid-private-key> \
  CANCEL_NOTIFY_SECRET=<any-random-string-used-as-internal-auth> \
  CRON_SECRET=<any-random-string-used-by-the-send-reminders-cron-job>
```

`SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` are auto-injected into every Edge Function by Supabase — you don't set them yourself.

---

3. Supabase Database Setup

1. Create a free project at supabase.com.
2. Go to **SQL Editor** and run each file in `supabase/migrations/` **in order** (001 → 010):

File	What it creates
`001_create_users.sql`	User profiles + roles
`002_create_courses.sql`	Course catalogue
`003_create_lecturer_courses.sql`	Lecturer → course assignments
`004_create_lectures.sql`	Lecture schedule
`005_create_enrollments.sql`	Student course enrollments
`006_create_attendance.sql`	Self-reported attendance
`007_create_tma_deadlines.sql`	TMA due dates
`008_create_notifications.sql`	In-app notification log
`009_create_app_settings.sql`	Semester config key-value store

File	What it creates
`010_rls_policies.sql`	Row Level Security for all tables

3. Go to **Database → Replication** and enable Realtime for the `lectures` and `notifications` tables.

---

4. Seeding the Admin Account

After running migrations, run `supabase/seed.sql` in the SQL Editor. It inserts:

- Sample courses (CIT 401, CIT 403, GST 101, etc.)
- Default semester settings (active session, facilitation dates)

Then manually create an admin user:

1. Go to **Authentication → Users → Add user** in Supabase dashboard.
2. Create the account (e.g. `admin@noun.edu.ng`).
3. In the SQL Editor, run:

```
UPDATE public.users
SET role = 'admin', status = 'active', onboarding_complete = true
WHERE email = 'admin@noun.edu.ng';
```

This gives that account full admin access when they log in.

---

5. VAPID Keys Push Notifications

Generate VAPID keys once and store them permanently:

```
npx web-push generate-vapid-keys
```

Output:

```
Public Key:  BHdj7...
Private Key: abc123...
```

- Put `Public Key` → `VITE_VAPID_PUBLIC_KEY` in `.env` and `VAPID_PUBLIC_KEY` in `supabase secrets set`
- Put `Private Key` → `VAPID_PRIVATE_KEY` in `supabase secrets set` only (never expose to the browser)

Important: If you ever regenerate these keys, all existing push subscriptions stored in `users.push_subscriptions` become invalid. Students must re-subscribe.

---

6. Running Locally

```
npm run dev
```

The app starts at `http://localhost:5173`.

> Push notifications require HTTPS so they won't fire on localhost. Deploy to a staging URL to test them end-to-end.

The two HTTP-triggered Edge Functions (`cancel-notify`, `create-lecturer`) are called directly at `${VITE_SUPABASE_URL}/functions/v1/<name>` — the same URL in dev and production — so there's no local proxy to run. Deploy a function after editing it (`npx supabase functions deploy <name>`) to test your change. If you want faster local iteration with hot reload instead, run `npx supabase functions serve` and point `VITE_SUPABASE_URL` at your local Supabase instance while developing.

---

7. Deployment

Netlify (static hosting only)

1. Push the repo to GitHub.
2. On netlify.com, click **Add new site → Import from Git**.
3. Set **Build command:** `npm run build` and **Publish directory:** `dist`.
4. Add the `VITE_*` environment variables from Section 2 under **Site Settings → Environment variables**.
5. Deploy.

The `netlify.toml` only configures the SPA redirect (all routes → `/index.html`) — Netlify no longer runs any backend functions.

Supabase (backend functions + cron)

1. Deploy all three Edge Functions: `npx supabase functions deploy send-reminders cancel-notify create-lecturer`.
2. Set the Edge Function secrets from Section 2: `npx supabase secrets set ...`.
3. In the Supabase Dashboard, go to **Integrations → Cron** and create a new job targeting the `send-reminders` Edge Function on schedule `*/5 * * * *`, with an `x-cron-secret` header matching your `CRON_SECRET`. This replaces what used to be Netlify's `[functions.send-reminders] schedule` config.

8. How Each Role Operates the App

Admin

Login: Use the admin credentials created in Section 4. After login you are redirected directly to `/admin` you never see the student dashboard.

Admin Panel (`/admin`) has six tabs:

Tab	What you do here
Overview	Live stat cards (Lectures, Courses, Students, Pending approvals); quick-action buttons; upcoming lectures this week; course distribution by level
Lectures	Create / edit / delete lectures; bulk import via CSV
Courses	Add courses (code, title, credit units, colour)
TMA	Set TMA 1, 2, 3 due dates per course
Users	View all users; suspend or restore accounts
Settings	Approve pending lecturers + assign them courses; update active semester and facilitation dates

Overview tab at a glance:

- Four coloured stat cards click **Lectures** to jump to the Lectures tab, click **Courses** to jump to Courses, click **Pending** to jump to Settings for approvals.
- A yellow warning banner appears automatically when lecturer accounts are waiting for approval.
- Quick action buttons let you open **New Lecture**, **New Course**, **Set TMA Deadline**, or **Manage Users** without switching tabs first.
- The **Upcoming This Week** panel lists the next 5 scheduled lectures with course code, facilitator, and time.
- The **Courses by Level** chart shows course counts for 100 / 200 / 300 / 400 level as proportional bars.

Creating a lecture:

1. Go to **Lectures → New Lecture** (or click **New Lecture** on the Overview quick-actions bar).
2. Pick the course and facilitator from the dropdowns.
3. Set start and end time.

4. Paste the Zoom URL the app auto-extracts the meeting ID and generates the deep link.
5. Save. The lecture appears instantly in enrolled students' calendars via Supabase Realtime.

Bulk import via CSV:

1. Click **Bulk Import → Download CSV template** to get the correct column format.
2. Fill in your data and upload the file.
3. Preview the rows, then click **Import All**.

Approving a lecturer:

1. Go to **Settings → Pending Lecturer Approvals** (or click the **Pending** stat card on the Overview, then scroll down).
2. Click **Approve** next to the lecturer's name.
3. Tick the courses they teach.
4. Click **Approve & Assign** the lecturer can now log in and manage their sessions.

Setting a TMA deadline:

1. Go to **TMA → Set TMA Deadline** (or click **Set TMA Deadline** on the Overview quick-actions bar).
2. Select the course, TMA number (1/2/3), and due date.
3. Save. A gold TMA block appears in enrolled students' calendars. Push reminders fire at 24 h and 1 h before the deadline.

Updating semester settings:

1. Go to **Settings → Semester Settings**.
2. Change the active academic session (e.g. `2026_1`), active semester, and facilitation start/end dates.
3. Save. A "New Semester" banner will appear for all students prompting them to re-enrol.

Admin Settings page (`/settings`):

- Edit your display name.
- Your email address is shown read-only (it is the admin login and cannot be changed here).
- Toggle notification types and enable/disable push for your device.
- Sign out.

Lecturer

Registering: Self-register at `/register` with role **Lecturer**. The account is locked (`status: pending`) until the Admin approves it and assigns you courses. You will see a holding page until approval is granted.

Lecturer Panel (`/lecturer`) has two tabs:

Tab	What you do here
Schedule	View upcoming assigned lectures; reschedule, cancel, post announcements, add recordings
Announcements	Review all announcements you've posted; edit them

Rescheduling a lecture:

1. Find the lecture in **Schedule**.
2. Click **Reschedule**, pick the new start and end time, and confirm.
3. All enrolled students receive an instant push notification with the new time.

Cancelling a lecture:

1. Click **Cancel** on the lecture card.
2. Type a reason (e.g. *Facilitator is ill session rescheduled to next Thursday*).
3. Confirm. Enrolled students are notified immediately. The lecture block turns red with a strikethrough in the calendar.

Posting an announcement:

1. Click **Announce** on an upcoming lecture card.
2. Type a message (max 280 characters).
3. Click **Post Announcement**. Students see a gold strip on the lecture block and receive a push notification.

Adding a recording link:

1. After a session ends, click **Recording** on the lecture card.
2. Paste the Zoom recording URL.
3. Save. Students receive a "Recording available" push and can open it from the lecture detail sheet.

Lecturer Settings page (`/settings`):

- Edit your **Full Name**, **Faculty**, and **Department** these are the details students and admins see

against your lectures.

- Your account status badge (Active / Pending) is shown read-only.
- Toggle notification types and enable/disable push for your device.
- Sign out.

Student

Registering: Self-register at `/register` with role **Student**. No admin approval needed students go straight to onboarding.

Onboarding Wizard (4 steps):

Step	What happens
1 Programme & Level	Select your programme (e.g. B.Sc. Computer Science) and level (100–500)
2 Course Registration Slip	Upload your PDF course reg slip PDF.js extracts your course codes automatically and shows a pre-ticked list for confirmation. If your slip is a scanned image, use manual selection filtered by programme and level
3 Push Notifications	Grant browser permission for push notifications
4 Install App	Install the PWA on your home screen for offline access

Dashboard (`/`):

- **Next Up card** shows the next lecture or TMA deadline with a live countdown. Displays a **Join Now** button 30 minutes before start.
- **Today's Schedule** all lectures and TMA deadlines for today, sorted by time.
- **Week Strip** 7-day overview with coloured dots showing lecture and TMA counts per day.

Calendar (`/calendar`):

- Switch between **Month**, **Week**, and **Day** views.
- Tap any lecture block to open the **detail sheet**:
    - Countdown timer (disappears after start)
    - **Join Meeting** button (deep link → web URL fallback)
    - Meeting ID and copy link
    - **Add to Calendar** (.ics download)

    - WhatsApp share
    - Announcement banner (if the lecturer posted one)
    - Recording link (after the session)
    - Attendance controls (after the session ends)
- TMA deadline blocks appear in gold tap to see the course name and due date.
- Cancelled lectures show in red with strikethrough.
- Rescheduled lectures show in orange.
- **LIVE** badge pulses green on a lecture that is currently running.

Attendance Self-Tracking:

1. After a lecture's end time has passed, open the lecture detail sheet.
2. Tap **I attended** (green), **I missed this** (red), or **Excused** (grey).
3. Optionally add a short note.
4. Your attendance summary (e.g. *8 attended / 2 missed / 12 total*) updates on the **Courses** page.

Courses (`/courses`):

- Lists every course you are enrolled in, with its colour, credit units, and attendance summary + progress bar.
- Tap **Enroll in more courses** to open the enrollment modal.

Notifications (bell icon in the top bar):

- All push events (reminders, cancellations, reschedules, announcements, recordings, TMA deadlines) also appear here as in-app notifications.
- Tap a notification to jump to the relevant lecture or TMA.
- **Mark all read** clears the badge counter.

Student Settings page (`/settings`):

- Edit your **Full Name**, **Matric Number**, **Programme** (dropdown), **Level** (dropdown), and **Study Centre** (dropdown).
- Toggle individual notification types (reminders / announcements / TMA deadlines).
- Enable or disable push notifications for this device.
- **Re-upload Course Registration Slip** tap this to restart the slip-upload step of onboarding if your enrolled courses have changed.
- Install the PWA (if not already installed).
- Sign out.

---

9. Push Notification Flow

Automated (Supabase Cron, every 5 minutes)

Trigger	Window	Message
30-min lecture reminder	28–32 min before start	"CIT 403 starts in 30 minutes Dr. Yusuf"
15-min lecture reminder	13–17 min before start	"CIT 403 starts in 15 minutes Tap to join"
5-min lecture reminder	3–7 min before start	"CIT 403 is about to start" + Join Zoom action
TMA 24h reminder	23–25 h before due	"TMA 1 for CIT 403 due tomorrow"
TMA 1h reminder	55–65 min before due	"TMA 1 for CIT 403 due in 1 hour submit NOW"

Instant (triggered by lecturer or admin actions)

Action	What students receive
Lecture cancelled	"CIT 403 (2pm) cancelled [reason]"
Lecture rescheduled	"CIT 403 rescheduled from 2pm → 4pm"
Recording added	"Recording available: CIT 403 Thursday session"
Announcement posted	"CIT 403 update Dr. Yusuf: [message]"

Expired subscriptions (HTTP 410) are automatically removed from the database.

---

10. PWA Installation

Android Chrome:

1. Open the app in Chrome.
2. Tap the browser menu → **Add to Home screen**.
3. Confirm. The app icon appears on your home screen and launches fullscreen.

iOS Safari (iOS 16.4+):

1. Open the app in Safari.
2. Tap the Share button → **Add to Home Screen**.

3. Confirm.

   Push notifications on iOS require iOS 16.4 or later and the app must be installed as a PWA.

The app caches the semester schedule via a service worker the calendar remains readable even without an internet connection. An **Offline** banner appears in the top bar when connectivity is lost.

---

11. Folder Structure

```
smart-lecture-reminder/
├── public/
│   ├── manifest.json          # PWA manifest
│   ├── sw-push.js             # Push notification service worker handler
│   └── icons/                 # PWA icons (72 → 512px)
│
├── src/
│   ├── components/
│   │   ├── ui/                # 15 pure-Tailwind primitives (no component library)
│   │   ├── layout/            # AppShell, Sidebar, Topbar, MobileNav
│   │   ├── auth/              # Login, Register, ForgotPassword, ProtectedRoute
│   │   ├── calendar/          # MonthView, WeekView, DayView, LectureBlock,
TMABlock, DetailSheet
│   │   ├── dashboard/         # NextUpCard, TodaySchedule, WeekStrip
│   │   ├── courses/           # CourseList, CourseCard, AttendanceSummary
│   │   ├── attendance/        # AttendanceControls, AttendanceNote
│   │   ├── notifications/     # NotificationBell, NotificationPanel,
NotificationItem
│   │   ├── onboarding/        # 4-step wizard (programme → slip upload → push →
install)
│   │   ├── admin/             # AdminDashboard + all admin sub-forms
│   │   ├── lecturer/          # LecturerDashboard +
reschedule/cancel/announce/recording modals
│   │   └── common/            # ErrorBoundary, UpdateBanner, OfflineBanner,
SearchBar
│   │
│   ├── contexts/              # AuthContext, CalendarContext, LecturesContext,
NotificationContext
│   ├── hooks/                 # useAuth, useCalendar, useLectures, useAttendance,
usePWAInstall, etc.
│   ├── services/              # supabase.js + all service modules (auth, lectures,
push, etc.)
│   ├── pages/                 # DashboardPage, CalendarPage, CoursesPage,
AdminPage, LecturerPage, SettingsPage
│   ├── utils/                 # dateHelpers, zoomHelpers, slipParser, csvParser,
formatters
│   └── constants/             # routes.js, reminderIntervals.js
│
├── supabase/
│   ├── config.toml            # Supabase CLI project config
│   ├── migrations/            # 001–013 SQL migration files
│   ├── seed.sql               # Sample courses + default semester settings
│   ├── scripts/               # reset-demo.mjs, seed-courses.mjs, expand-
lectures.mjs
│   └── functions/
│       ├── _shared/           # cors.ts, supabaseAdmin.ts, push.ts
```

```
│       ├── send-reminders/    # Cron: lecture + TMA push reminders (runs every 5
min)
│       ├── cancel-notify/     # HTTP: instant push for
cancel/reschedule/announce/recording
│       └── create-lecturer/   # HTTP: admin bulk lecturer import
│
├── tests/
│   └── e2e/
│       └── pw-test.mjs        # Playwright smoke-test suite
(admin/lecturer/student/cross-role)
│
├── netlify.toml               # Build config + SPA redirect only
├── vite.config.js             # Vite + PWA plugin config
└── tailwind.config.js         # NOUN green palette + font config
```

---

12. Known Limitations

- **iOS push before 16.4:** Web Push is not supported on iOS Safari prior to version 16.4. Students on older iOS receive in-app notifications only they still see the notification bell panel.
- **Offline write:** The service worker caches the calendar for offline reading, but creating or updating data (attendance marks, etc.) requires a connection.
- **Zoom deep links:** The `zoommtg://` deep link requires the Zoom desktop or mobile app to be installed. If it is not, the app falls back to the browser Zoom URL automatically.
- **PDF scanned slips:** PDF.js can only extract text from digitally-generated PDFs. Scanned image-based registration slips require manual course selection.